CATEGORY C

Coding Assignment

Problem 1: Movie Ticket Counter

TASK

A cinema sells three kinds of seats: regular, premium, and recliner. Each seat type has its own ticket price, but every ticket — whatever the seat type — also carries the same convenience fee of 20. The fee must be written in one place only.

The counter needs the amount for each booking and the total collected. A plain "ticket" with no seat type must never be sold.

INPUT / OUTPUT

Input: The first line contains N, the number of bookings. Each of the next N lines contains `SEAT count`, where SEAT is REGULAR, PREMIUM, or RECLINER and count is the number of tickets.

Output: For each booking, print `SEAT: amount`. Finally, print `Total: total`. All values are formatted to two decimal places.

BUSINESS RULES

- Regular seat: 150 per ticket.
- Premium seat: 250 per ticket.
- Recliner seat: 400 per ticket.
- Every ticket adds a convenience fee of 20.

EXAMPLE

Sample Input

```
3
REGULAR 3
PREMIUM 2
RECLINER 1
```

Expected Output

```
REGULAR: 510.00
PREMIUM: 540.00
RECLINER: 420.00
Total: 1470.00
```

HINT

> *Think about what every ticket has in common (the count and the convenience fee) and what changes from one seat type to another (the price). Where should the part that never changes be written so that it exists only once?*

STARTER CODE

```
// Implement: Movie Ticket Counter
```

Problem 2: Parcel Shipping Desk

TASK

A courier office ships standard, express, and fragile parcels. Every parcel has a weight and a declared value, and each parcel type calculates its shipping charge differently. Express and fragile parcels can also be insured; standard parcels cannot.

The office needs the charge, the insurance, and the total for each parcel, followed by the grand total. Making another parcel type insurable later should not change the parcel types that are not insured.

INPUT / OUTPUT

Input: The first line contains N. Each of the next N lines contains `TYPE weightKg declaredValue`, where TYPE is STANDARD, EXPRESS, or FRAGILE.

Output: For each parcel, print `TYPE: Charge=c Insurance=i Total=t`. Finally, print `Grand Total: g`. All values are formatted to two decimal places.

BUSINESS RULES

- Standard: 40 plus 10 per kg.
- Express: 80 plus 15 per kg.
- Fragile: the standard charge plus a handling fee of 50.
- Insurance for express and fragile parcels is 2% of the declared value. Standard parcels show `Insurance=0.00`.

EXAMPLE

Sample Input

```
3
STANDARD 3 500
EXPRESS 2 1000
FRAGILE 4 2000
```

Expected Output

```
STANDARD: Charge=70.00 Insurance=0.00 Total=70.00
EXPRESS: Charge=110.00 Insurance=20.00 Total=130.00
FRAGILE: Charge=130.00 Insurance=40.00 Total=170.00
Grand Total: 370.00
```

HINT

> *Two different questions are being asked of each parcel: "how much does shipping cost?" (every parcel answers this) and "can it be insured?" (only some parcels can). Consider whether these two questions belong in the same place in your design.*

STARTER CODE

```
// Implement: Parcel Shipping Desk
```

Problem 3: College Fee Counter

TASK

A college collects yearly fees from three kinds of students: day scholars, hostellers, and scholarship students. Every student has a name and pays tuition, but the amount and the extra charges differ. Day scholars and scholarship students use the college bus; hostellers live on campus and do not.

The fee counter needs each student's total fee and the total collected.

INPUT / OUTPUT

Input: The first line contains N. Each of the next N lines contains `TYPE Name`, where TYPE is DAY_SCHOLAR, HOSTELLER, or SCHOLAR.

Output: For each student, print `Name: fee`. Finally, print `Total Collected: total`, formatted to two decimal places.

BUSINESS RULES

- Day scholar: tuition 40000.
- Hosteller: tuition 40000 plus hostel fee 60000.
- Scholarship student: half of the normal tuition (20000).
- Every student who uses the college bus pays a transport fee of 12000.

EXAMPLE

Sample Input

```
3
DAY_SCHOLAR Asha
HOSTELLER Ravi
SCHOLAR Neha
```

Expected Output

```
Asha: 52000.00
Ravi: 100000.00
Neha: 32000.00
Total Collected: 184000.00
```

HINT

Every student has a name and a tuition rule, but only some students use the bus. Keep the transport fee in one place, and think about how the fee counter can find out whether a student uses the bus without checking the student type by name.

STARTER CODE

```
// Implement: College Fee Counter
```

Problem 4: City Cab Fare Meter

TASK

A cab company runs three kinds of cabs: mini, sedan, and SUV. Each cab type charges a different rate per kilometre, but the company has one rule for all cabs: no trip costs less than 100. Only sedans and SUVs offer night service, which adds 20% to the fare. A night trip requested in a mini must be rejected.

The company needs the fare for every completed trip and the total of all completed trips.

INPUT / OUTPUT

Input: The first line contains N. Each of the next N lines contains `CAB km TIME`, where CAB is MINI, SEDAN, or SUV and TIME is DAY or NIGHT.

Output: For each completed trip, print `CAB: fare`. For a rejected trip, print `CAB: night service not available`. Finally, print `Total: total`. All values are formatted to two decimal places.

BUSINESS RULES

- Mini: 10 per km. Sedan: 14 per km. SUV: 18 per km.

- Fare = distance × rate, but never less than 100.
- A night trip adds 20% to the fare (after the minimum is applied).

EXAMPLE

Sample Input

```
4
MINI 8 DAY
SEDAN 10 NIGHT
SUV 20 DAY
MINI 5 NIGHT
```

Expected Output

```
MINI: 100.00
SEDAN: 168.00
SUV: 360.00
MINI: night service not available
Total: 628.00
```

HINT

The minimum fare applies to every cab, while night service is an ability that only some cabs have. Where would you write the minimum-fare rule so it is shared, and how would you represent "can offer night service" so that a new cab type can choose to have it or not?

STARTER CODE

```
// Implement: City Cab Fare Meter
```

Problem 5: Home Appliance Energy Report

TASK

An energy app estimates the monthly electricity cost of four home appliances: a fridge, an air conditioner, a TV, and a washing machine. Every appliance has a fixed power rating and a number of hours used. The air conditioner and the washing machine also have a saver mode that reduces their energy use by 25%; the fridge and the TV do not.

The app must show the units used and the cost for each appliance, and the total cost. If saver mode is requested for an appliance that does not support it, the app must report it instead of calculating a cost.

INPUT / OUTPUT

Input: The first line contains N. Each of the next N lines contains `APPLIANCE hours` or `APPLIANCE hours SAVER`, where APPLIANCE is FRIDGE, AC, TV, or WASHER.

Output: For each appliance, print `APPLIANCE: Units=u Cost=c`, or `APPLIANCE: saver mode not supported`. Finally, print `Total Cost: total`. All values are formatted to two decimal places.

BUSINESS RULES

- Power ratings: Fridge 150 W, AC 1500 W, TV 100 W, Washer 500 W.
- Units (kWh) = power × hours ÷ 1000. In saver mode, units are reduced by 25%.
- Cost = units × 8.

EXAMPLE

Sample Input

```
4
FRIDGE 24
AC 8 SAVER
TV 5
WASHER 2 SAVER
```

Expected Output

```
FRIDGE: Units=3.60 Cost=28.80
AC: Units=9.00 Cost=72.00
TV: Units=0.50 Cost=4.00
WASHER: Units=0.75 Cost=6.00
Total Cost: 110.80
```

HINT

Every appliance calculates units the same way from its own power rating, but only some appliances have a saver mode. Think about what all appliances share, what only some of them can do, and how the report can ask "does this appliance support saver mode?" without a list of if-else checks.

STARTER CODE

```
// Implement: Home Appliance Energy Report
```